

Programovacie jazyky

1. Zoznamy, n-tice, algebraické dátové typy (ADT), zovšeobecnené algebraické dátové typy (GADT), abstraktné dátové typy a moduly

- Zoznamy:
 - `data [] a = [] | a:[a]`
 - `head, init` su $O(1)$
 - `tail, last, length, (!!)` su $O(n)$
 - `map, zip`
- n-tice:
 - Mozu mať rozne typy
 - `(a, b, c) = (True, 1, "xyz")`
 - `fst, snd`
- Algebraicke datove typy (ADT):
 - Su to typy, ktore vzniknu skladanim inych typov (composite)
 - Product types (zaznamy, n-tice):
 - `data Student = MkSt { name :: String, age :: Int }`
 - Student je typovy konstruktor a `MkSt` je datovy konstruktor
 - Sum types (union):
 - `data Bool = True | False`
 - `data Maybe a = Nothing | Just a`
 - Rekurzivne typy:
 - `data List a = Empty | Cons a (List a)`
- Zovseobecnene algebraicke datove typy (GADT):
 - Umoznuju obmedzit navratovu hodnotu konstruktora
 - Pri ADT musi byt navratova hodnota `Expr a`
 - Pri GADT moze byt `Expr Int`, alebo `Expr String`
 - Napriklad ked mame datovy konstruktora `Add` pre `Expr`, tak moze byt `Add :: Expr Int -> Expr Int -> Expr Int`, cize ked spravime `Add (I 10) (B True)`, tak to bude chyba
- Moduly:
 - Modul je zoskupenie suvisiacich funkcií, typov, typovych tried, ...
 - Program je subor modulov
 - Moduly mozu importovat ine moduly
 - Program sa spusta zavolaním `main` z modulu `Main`
 - `module Stack (push, pop, top) where ...`
- Abstraktne datove typy:
 - Datovy typ spolu s operaciami, pričom sa ukryvajú implementačné detaily
 - Dostupne operácie = rozhranie
 - Napríklad `Data.Map`, alebo `Data.Set`
 - Ak zachovame rozhranie, implementáciu môžeme meniť

2. Pattern matching a kontrola jeho úplnosti, chvostová rekúzia, akumulátor, typy ntého ranku (Rank-N types).

- Pattern matching a kontrola jeho úplnosti
 - Možnosti:
 - viac definícií pre jednotlivé paterny
 - guards
 - case
 - Catch-all pattern: `_`
 - Anonymne premenne: `_` (može sa opakovane použiť)
 - Na neuplne vzory: `-Wincomplete-patterns`
 - Aliasing (as-pattern): `@`
- chvostová rekúzia, akumulátor
 - Volané `g` vo funkcii `f` je chvostové, keď je to posledné volanie a teda výsledkom `f` je volanie `g`
 - Keď všetky rekúzivné volania sú chvostové, tak je to chvostová rekúzia
 - Da sa znovupoužiť aktivacný záznam na stacku
 - Rekúzia sa dá prepísať na chvostovú akumulátorom:
 - `fib curr prev 0 = curr`
 - `fib curr prev n = fib (curr + prev) curr (n-1)`
 - Da sa prepísať aj cez continuation passing style
 - Ze si posiela namiesto akumulátora funkciu, ktorú postupne buduje a v base case zavola na 0
 - Velké thunky
- typy ntého ranku (Rank-N types).
 - Majme funkciu:
 - `f :: (a -> a) -> a -> a`
 - `f g x = g x`
 - Tá funkcia `g` nie je polymorfická! Ak by sme ju v tele `f` zavolali na dve hodnoty rôznych typov (cize `g "xyz"`, `g 1000`), tak by to nefungovalo. Lebo podľa toho `x` sa instanciuje aj `g`.
 - Aby bola polymorfická a teda aby sme mohli v tele robiť aj `g "xyz"`, aj `g 1000`, tak spravíme napríklad:
 - `f :: (forall a. Show a => a -> a) -> String`
 - `f g = show (g 1) ++ show (g "xyz")`
 - Typ ranku `n` je funkcia, ktorá má aspoň jeden argument ranku `n-1`, ale žiaden argument vyššieho ranku
 - System F (ranku `n`) vs Hindley-Milnerov (iba ranku `1`) typový systém

3. Striktné (strict), odložené (delayed) a lenivé (lazy) vyhodnocovanie parametrov. Odloženie vyhodnotenia parametrov v striktných jazykoch (thunks). Vynútenie vyhodnotenia parametrov v lenivých jazykoch (foldl vs. foldl'). Meniteľné (mutable) a nemenné (immutable) dátové štruktúry.

- Striktné (strict), odložené (delayed) a lenivé (lazy) vyhodnocovanie parametrov.
 - Striktne:
 - Weak-head normal form (WHNF):
 - Ak najvonkajšia časť výrazu je jedno z:
 - konštruktor
 - lambda abstrakcia ($\lambda x \rightarrow \text{výraz}$)
 - neuplná aplikácia vstavanej funkcie (napr. (+) 2)
 - Normal form:
 - Keď je výraz plne vyhodnotený (každý podvýraz)
 - Striktne vyhodnocovanie hneď vyhodnocuje parametre
 - Do Haskellu sa dá dať cez seq
 - seq zoberie 2 argumenty a vráti druhý, ale ako side-effect vyhodnotí prvý do WHNF
 - $f \$! x = x \text{ 'seq' } f x$
 - Cize vyhodnotí x a potom ho dá do f už vyhodnotene
 - Pre NF je deepseq
 - Delayed:
 - Argumenty sa dávajú nevyhodnotené
 - Vyhodnotia sa pri každom použití (takže možno viackrát)
 - Lazy:
 - Argument sa dáva nevyhodnotený
 - Keď sa už vyhodnotí, tak sa memoizuje
 - Asi nie ideálne keď to má side-effekty?
- Odloženie vyhodnotenia parametrov v striktných jazykoch (thunks).
 - Telo funkcie sa nevyhodnocuje až do jej zavolania.
 - Môžeme teda spraviť " $() \rightarrow x + y$ ", čím sa nevyhodnotí $x + y$ až do zavolania, cize spravíme thunk pre $x + y$
 - Vždy sa to bude vyhodnocovať znova, takže chceme memoizovať vyhodnotenie:
 - Budeme mať dvojicu (false, thunk) pre ešte nevyhodnotený a (true, value) pre už vyhodnotený argument. Bude to mutovateľná dvojica, cize sa to prepíše a vždy sa použije (true, value) keď už to bude prepísané
- Vynútenie vyhodnotenia parametrov v lenivých jazykoch (foldl vs. foldl').
 - foldl' sa dá pomocou seq, že vždy akumulátor najskôr vyhodnotíme a až potom rekurzívne zavoláme foldl'
 - $\text{foldl}' f \text{ acc } (x:xs) = \text{let } z = f \text{ acc } x \text{ in } z \text{ 'seq' } \text{foldl}' f z xs$
- Meniteľné (mutable) a nemenné (immutable) dátové štruktúry.
 - Immutable:
 - Nedajú sa meniť
 - Časti sa zdieľajú

- Thread-safe
- Mutable:
 - V haskelli:
 - Je na to RunST monada

4. Návrhový vzor monad a jeho využitie (Maybe, List, Reader, Writer, State).

- Návrhový vzor monad a jeho využitie (Maybe, List, Reader, Writer, State).
 - Funktor
 - `fmap :: (a -> b) -> f a -> f b`
 - `fmap = <$>`
 - cize aplikuje funkciu v kontexte (Maybe, list, Either, ...)
 - Aplikativny funktor
 - Co ked chceme aplikovat binarnu funkciu na 2 rozne hodnoty v kontexte?
 - Cez fmap dostaneme po prvej aplikacii funkciu v kontexte (ciastocne aplikovanu) a nemozeme to uz aplikovat na druhu vec fmapom
 - `<*> :: f (a -> b) -> f a -> f b`
 - Cize ako fmap, ale ta funkcia je v kontexte
 - Monada:
 - `(>>=) :: m a -> (a -> m b) -> m b`
 - `return`
 - Vieme s monadami spravit:
 - `productsM :: [Integer]`
`productsM = do`
`x <- [1..3]`
`y <- [1..x]`
`return (x * y)`
 - Ale s aplikativnymi funktormi to nevieme:
 - `productsA = (*) <$> [1..3] <*> [1..???`
 - Porovnanie:
 - Funktor nevie zmenit kontext, lebo nevie nic o kontexte.
 - Aplikativny funktor moze menit, kontext, lebo funkcia je v kontexte, ale vnutri je cista funkcia, takže tvar kontextu nemoze menit. Napríklad `<*>` na zoznam dlzky 10 vrati vzdy zoznam dlzky 10.
 - Monada moze menit tvar vysledneho kontextu, lebo kvoli typu `a -> m b` vie nastavovat kontext podľa hodnoty
 - `(\x -> replicate x x) =<< [1..4]`
 - `= [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]`
 - Pocet prvkov vysledku nezalezi len od funkcie na lavej strane, ale aj od cistych hodnot na pravej strane

- Monada je aplikativny funktor a aplikativny funktor je funktor
- Maybe, List, Reader, Writer, State
 - Maybe:
 - `data Maybe a = Nothing | Just a`
 - Na vypocty, ktore mozu zlyhat, ked nechceme explicitny error checking
 - Optional
 - List:
 - `return x = [x]`
 - `xs >>= f = concat (map f xs)`
 - ```
do { x1 <- action1
 ; x2 <- action2
 ; mk_action3 x1 x2 }
```
    - Je to iste ako:
    - `action1 >>= (\ x1 -> action2 >>= (\ x2 -> mk_action3 x1 x2 ))`
  - Reader:
    - Je to ako "dependency injection"
    - Mozeme ako argument dat Reader monadu, odkial moze funkcia citat read-only stav (**ask**).
  - Writer:
    - To je uzitocne pre funkcie, ktore maju okrem vratenia hodnoty aj produkovať logy. Writer monada akumuluje tieto logy cez bind (**tell**)
  - State:
    - Na posuvanie stavu. Existuje **runState**, cim zacne vypocet so stavom. On dostane pociatocny stav a vypocet. Ten vypocet je nejaka seria bindov funkcií, ktore beru aj stav a modifikuju ho.
    - Vie robit **get** a **put**.

## 5. Prvorádové (first-class) funkcie, funkcie vyššieho rádu (map, foldr, foldl), uzávery (closures), typová inferencia.

- Prvorádové (first-class) funkcie
  - Neberu funkciu ako argument
  - Prikklady: `sum`, `product`, `zip`, `words`, `concat`, ...
- funkcie vyššieho rádu (`map`, `foldr`, `foldl`)
  - Beru funkciu ako argument
  - `map`:
    - `map :: (a -> b) -> [a] -> [b]`
  - `foldr`:
    - `foldr :: (a -> b -> b) -> b -> [a] -> b`

- $\text{foldr} :: \text{Foldable } t \Rightarrow (a \rightarrow b \rightarrow b) \rightarrow b \rightarrow t \ a \rightarrow b$
    - Implementacia:
      - $\text{foldr } _ e [] = e$
      - $\text{foldr } f e (x:xs) = f x (\text{foldr } f e xs)$
    - $x_1 \oplus (\dots (x_{n-1} \oplus (x_n \oplus e)) \dots)$
    - nepouziva teda chvostovu rekurziu
  - foldl:
    - $\text{foldl} :: \text{Foldable } t \Rightarrow (b \rightarrow a \rightarrow b) \rightarrow b \rightarrow t \ a \rightarrow b$
    - Implementacia:
      - $\text{foldl } _ e [] = e$
      - $\text{foldl } f e (x:xs) = \text{foldl } f (f e x) xs$
    - $(\dots ((e \otimes x_1) \otimes x_2) \dots) \otimes x_n$
    - ma vela odlozenych medzivypoctov
  - foldl':
    - foldl, ale s priebeznym vyhodnocovanim akumulatora
  - Pre striktné funkcie pouzivat foldl', pre ostatne foldr (napríklad (:))
  - Ak  $(S, +, e)$  je monoid ( $S$  je mnozina,  $+$  je operacia,  $e$  je neutralny prvok), tak  $\text{foldl} = \text{foldr}$
  - $\text{foldr } f e xs = \text{foldl } (\text{flip } f) e (\text{reverse } xs)$
  - $\text{foldl1}$ ,  $\text{foldr1}$ : pociatocna hodnota akumulatora je 1. prvok
  - scan je ako fold, ale vracia zoznam vsetkych medzivysledkov
- uzavery (closures)
    - Ak nieco defunujeme pomocou premennej, ktora medzi definiciou a pouzitim zanikne, tak to stale bude fungovat.
    - Dynamicky scoping nepouziva closures (vzdy berie zo stacku)
    - Priklad s "const x = new PrivateContainer(12)" -> ta 12 bude v closure
      - Podstatne je pouzit funkciu na odloženie private premennych
  - typová inferencia.
    - Typy premennych sa mozu dat odvodiť
      - C++: auto x = foo(1, 2)
      - C#: var myOwnVar = new { Name = "John", Age = 100 };
        - Vytvori novy typ a inferuje ho

## 6. Funkcie viacerých argumentov, n-tice, currying vs. čiastočné aplikovanie funkcie, bezbodový (pointfree) štýl zápisu funkcie.

- Funkcie viacerých argumentov
  - Haskell ma len unarne funkcie
- n-tice
  - Moze byt unarna funkcia beruca n-ticu namiesto n-arnej funkcie

- currying vs. čiastočné aplikovanie funkcie
  - Currying:
    - Transformovanie n-arnej funkcie na postupnosť unarných
    - $\text{curry} :: ((a, b) \rightarrow c) \rightarrow (a \rightarrow b \rightarrow c)$
    - $\text{uncurry} :: (a \rightarrow b \rightarrow c) \rightarrow ((a, b) \rightarrow c)$
  - Čiastočné aplikovanie funkcie
    - Keďže Haskell má len unarne funkcie, tak sa dajú čiastočne aplikovať
- bezbodový (pointfree) štýl zápisu funkcie.
  - V ňom sa nespomínajú argumenty
  - $\text{sum xs} = \text{foldl}' (+) 0 \text{ xs} \rightarrow \text{sum} = \text{foldl}' (+) 0$
  - Da sa spraviť pomocou:
    - currying
    - skladanie funkcií
    - eta-redukcia
    - konverzia na prefixový zápis operátora
  - Príklad:  $\text{any p list} = \text{or (map p list)}$

## 7. Staticky a dynamicky typované jazyky, jazyky bez typovej kontroly, možné rozdiely v sémantike príkazu if a zoznamov, duck typing.

- Staticky a dynamicky typované jazyky
  - Statická kontrola:
    - Ma zamietnuť zlý program ešte pred spustením
    - Nevie detegovať logické chyby (preklep v názve premennej, delenie 0)
    - Je **korektná**, ak nedáva žiadne false negatives
    - Je **uplná**, ak nedáva žiadne false positives
    - Typy ranku n majú za cieľ zvýšiť úplnosť, lebo bez nich Haskell nepuští:
      - $f\ x = (x, 1)$
      - $g\ f\ x\ y = (f\ x, f\ y)$
      - $g\ f\ \text{True}\ 'a'$
    - Nemože súčasne splnať:
      - úplnosť, korektnosť, skončenie v konečnom case
    - Môžeme teda pridať dynamické kontroly
    - Alebo niečo prehlásime za nedefinované správanie
      - To nazývame slabá kontrola
  - Dynamická kontrola
    - Nie vždy sa robí (lebo to spomaľuje runtime)
    - Môže to byť príjemné (zoznamy s rôznymi typmi, ľubovoľný nesting)
    - Netreba písať typy, kontrolujú sa za behu
      - Pri statickej musíme niektoré veci otypovať, je to pracnejšie

- Ale nemozeme predpokladat nic o typoch, musime rucne kontrolovat v programe
  - Da sa znovupouzit kod pre rozne typy
    - Ale staticke to vedia tiez (generics, dedenie, ...)
    - Tazko sa potom hladaju chyby
- Prototypovanie:
  - Pri dynamickej nemusime riesit vsetky pripady, proste ich neimplementujeme a nic sa nestazuje
  - Pri statickej ale staci dat **error "Not implemented"** a tym to vyriesime a naviac mame viac kontrol
- Udrzba:
  - Pri dynamickej volajuci nemusi byt zmeneny, ked sa zmeni typ, ktorý očakáva funkcia (napríklad doteraz bol int a teraz bude int | string)
  - Pri statickej volajuci musí byť zmenený, keď sa zmení volaný (aby používal nový typ). Ale zase statická kontrola vráti zoznam miest, kde to treba zmeniť.
- jazyky bez typovej kontroly
  - Assembler nema typove kontroly
  - Programator musi vediet, co je co
- možné rozdiely v sémantike príkazu if a zoznamov
  - if:
    - TODO (možno konverzia podmienky na bool?)
  - zoznamy:
    - V staticky typovaných sú homogénne, v dynamicky heterogénne
- duck typing
  - Objekt je považovaný za nejakého typu, ak má interface toho typu. Čiže ak má všetky vlastnosti, čo má mať ten typ, tak to považujeme za ten typ.
    - Ak to vyzerá ako kacka, ...
  - Určuje sa to iba podľa toho, čo sa používa v runtime, čiže nemusí ten objekt splňať úplne všetko z toho typu, ktorým chce byť.

## 8. Funkcionálna vs. OOP dekompozícia, mixins, double dispatch, subtyping, generics, polymorfné funkcie

- Funkcionálna vs. OOP dekompozícia
  - Funkcionalna:
    - Rozbije systém na funkcie, tie ešte dekomponuje, aby zistila opakovanie medzi nimi a podľa toho sa postaví moduly.
    - Data sa presuvajú medzi funkciami, tieto data určujú stav programu.
    - calculate\_salary, process\_tax
  - OOP dekompozícia:

- Rozbije system na objekty a kazdemu objektu asociuje spravenie a data.
  - Data su v objektoch, to predstavuje stav systemu.
  - Objekty zodpovedaju "realnym" veciam
  - Employee, Salary, TaxAuthority
- Porovnanie:
  - Funkcionalna rozbija na mensie moduly, OOP na vacsie (a lepsie zrozumitelne osamote)
  - Funkcionalna rozdeluje data a funkcie. OOP spaja data a spravenie.
  - Pri funkcionalnej sa volaju funkcie. Pri OOP sa posielaju spravy medzi objektami.
- mixins
  - Mixin je trieda, ktora sa "injectuje" do inych tried. Napríklad mozeme mat mixin, ktory prida nejaku property, alebo funkciu. Potom to vieme "injectnut" do viacerých roznych tried.
  - Mixin nevie stat osamote, chce byt injectnuty.
  - Příklad:
    - V Java maju interfacys uz defaultne implementacie metod. Takyto interface je mixin.
  - Je to vzor na dependency inversion.
  - Nie su tam zlozite semanticke veci nasobneho dedenia.
- double dispatch
  - Ak to, aka funkcia sa zavola zavisí iba na objekte, na ktorom sa vola, tak je to single dispatch (dispatchne sa to volanie tomu objektu). To su napríklad virtualne funkcie (zavola sa spravny override)
  - Ak to, aka funkcia sa zavola zavisí aj od argumentu, tak je to double dispatch.
  - Příklad:
    - Handlovanie eventov. Nejaký objekt dostane event a ten event je nejakeho typu. Podľa typu eventu sa rozhodne ten objekt, ze ktory handler zavola.
    - class Rectangle:
 

```
def display_on(port):
 case port of:
 ...
```
    - Namiesto kejsovania sa da spravit:
    - class Rectangle:
 

```
def display_on(port):
 port.display_rectangle(self)
```
    - A potom iba jednotlivé classy pre porty implementuju kazda display\_rectangle, display\_oval, ...
- subtyping
  - Podtrieda je specializaciou nadtriedy
  - Liskov substitution principle
    - Objekt nadtriedy moze byt nahradeny objektom podtriedy a nic sa nepokazi.

- Polymorfizmus:
  - Rozne podtriedy mozu rozne prepisat rovnaku parent metodu
- Nemusi to zahrnat iba dedenie. Dedenie robi subtyping, ale subtyping je napríklad aj pouzivanie interfacov.
- kovariancia, kontravariancia, invariancia (A, B, su typy, f je napríklad List):
  - kovariancia:  $A < B \Rightarrow f A < f B$ 
    - Cize List<String> je podtypom List<Object>
  - kontravariancia:
    - naopak
  - invariancia:
    - ani jedno
    - v Java platí toto, cize List je invariantny
- generics
  - Aby mohla metoda fungovat aj na podtypoch / nadtypoch a stale aby to bolo staticky type-safe
  - trieda / metoda je generic, ak obsahuje aspon jeden typovy parameter
- polymorfické funkcie
  - parametricky polymorfizmus:
    - Pri definicii funkcie mozu byt pouzite typove parametre
    - Funkcia sa potom instanciuje podla parametrov
    - Vsetky instanciovane funkcie maju **rovnake spravanie**
    - templates, generics, haskell: polymorficke funkcie
      - $\text{map} :: (a \rightarrow b) \rightarrow [a] \rightarrow [b]$
  - ad-hoc polymorfizmus:
    - Pretazovanie metod, rozne pretazenia mozu mat **rozne spravanie**
    - overloading, haskell: typove triedy (typeclasses)
      - Eq, Enum, Bounded, Num, Show, ...

## 9. LISP syntax, rozdiel medzi reťazcami a symbolmi, eval, quasiquote, unquote, string interpolation

- LISP syntax
  - Vsetko je list
  - Racket:
    - definicia premennej:
      - (define x 42)
    - volanie funkcie (vždy prefixovy tvar, plne uzatvorenie):
      - (\* 4 2)
    - definicia funkcie:

- (define cube (lambda (x) (\* x x x)))
      - Syntakticky cukor: (define (cube x) (\* x x x))
    - viacargumentove funkcie (skutocne):
      - (define (objem a v) (\* a a v))
    - (if podmienka true-vyraz false-vyraz)
  - rozdiel medzi reťazcami a symbolmi
    - Zapisuje sa s ' na zaciatku, cize napriklad 'x
    - Symbol je struktura 5 pointrov (meno ako string, hash, hodnota, vlastnosti, next entry v hash tabulke v rovnakom buckete)
      - Porovnavanie symbolov je porovnavanie tych hashov
    - Porovnanie stringov je O(n)
  - eval
    - Umoznuje v runtime vykonat kod zo stringu.
    - Treba aby sucastou jazyka bolo to prostredie (interpreter / kompilator)
  - quasiquote, unquote
    - (quote v) je to iste ako 'v
      - Zabrani interpretovaniu zoznamov ako volaniu funkcii
    - (unquote v) je to iste ako ,v
      - Cize vyhodnoti v
    - (quasiquote v) je `v pracuje ako (quote v), ale ak narazi na (unquote x), tak vyhodnoti x a nahradi nim (unquote x)
      - V JavaScripte je na to string interpolation
        - `Hello \${name}`
  - string interpolation
    - V predch. odrazke

## 10. Makrá, kedy je ich vhodné použiť, hygienické makrá.

- Makrá, kedy je ich vhodné použiť, hygienické makrá.
  - Definícia makra popisuje, ako transformovať novú syntax do syntaxe pôvodného jazyka
  - Makro-systém je jazyk pre definíciu makier
  - Rozbalenie makier je prepísanie každého použitia makra
    - Makroprocesor
  - Příklad v racket (thunk):
    - (delay e) -> (dly (lambda () e))
  - V Rackete sa definujú pomocou define-syntax a syntax-rules
  - Problémy:
    - Tokenizácia (nechceme aby pracovali po znakoch, ale po tokenoch)
    - Uzatvorkovanie
      - #define MUL(x,y) x\*y

- $MUL(1 + 2, 3 + 4) * 5 \rightarrow 1 + 2 * 3 + 4 * 5$
  - Preto v C makra plne zatvorkujeme
  - Racket nema tento problem, lebo je plne zatvorkovany
- V C lokálne premenne nezatienujú makra
  - Preto je v C konvencia že makra sa pisu VELKYMI
  - Ale v Rackete zatienujú
    - Čiže sa lokálne premenne neprepisu makroprocesorom, ani keď sa volajú rovnako, ako názov nejakého makra
- Je rozdiel spraviť:
  - `(define (dbl x) (+ x x))`  
`(define (dbl x) (* 2 x))`
  - Lebo keď tomu dáme argument čo aj niečo printuje, tak v 1. prípade sa to printne 2-krát a v 2. iba raz.
    - Lebo sa argument vyhodnotí 2-krát
    - Da sa to riešiť tak, že v definícii makra najskôr argumenty dáme do nových lokálnych premenných a v tele makra pracujeme iba s nimi
- Problem s lokálnymi premennými:
  - Majme makro:
    - `(define-syntax dbl`  
`(syntax-rules ()`  
`[(dbl x)`  
`(let ([y 1]) (* 2 x y))])`
    - Rozbalme `(let ([y 7]) (dbl y))`:
      - `(let ([y 7]) (let ([y 1]) (* 2 y y)))`
      - Čiže sa to vonkajšie `y=7` prepíše tým makrovým lokálnym, ak sa to spravi naivne
      - Ale Racket nahradza tie lokálne premenne v makrach unikátnymi
        - 1. súčasť makro-hygieny v rackete
      - C má tento problém
    - Alebo majme makro:
      - `(define-syntax dbl`  
`(syntax-rules ()`  
`[(dbl x) (* 2 x)])`
      - Naivne rozbalenie `(let ([* +]) (dbl 42))`:
        - `(let ([* +]) (* 2 42))`
          - Čiže počíta namiesto násobenia
        - C má tento problém
        - Racket nie, on vyhľadáva hodnoty premenných podľa toho kde boli lexikálne definované a nie kde boli použité
          - 2. súčasť makro-hygieny v rackete